

实验报告

课程:机器学习	实验名称:决策树	日期:4.13
姓名:雷文豪	学号:138022024031	专业班级:人工智能

1 实验目的

1. 掌握决策树的构造和分类原理，以及使用决策树算法求解问题。
2. 学习解题设计、通过代码编写与运行实施解题、完成程序正确性的验证。

2 实验内容提要 with 实验设备

1. 给待解问题设计求解方法和适合的模型。
2. 对给定数据集，设计适合模型的求解方法、编写程序实施求解。
3. 实验结果的分析与确认。
4. 实验设备：个人电脑或便携式电脑，并安装 Windows 等操作系统和编程开发环境。

3 实验内容与步骤

3.1 信用卡交易的分类问题（模仿实例例题求解）

给定信用卡交易数据集，请构造一个分类决策树模型判别交易类别（欺诈、非欺诈），并评估该分类模型的泛化性能。

数据集每个样本的属性名称为 [Refund, MaritalStatus, Income, Cheat]，15 个样本如下：

```
['Yes', 'Single', 85, 'No'], ['No', 'Married', 105, 'No'], ['Yes', 'Married', 130, 'No'],  
['No', 'Single', 95, 'No'], ['No', 'Divorced', 85, 'Yes'], ['No', 'Married', 70, 'No'],  
['Yes', 'Divorced', 150, 'No'], ['No', 'Single', 80, 'Yes'], ['No', 'Married', 85, 'No'],  
['No', 'Single', 70, 'Yes'], ['No', 'Married', 75, 'No'], ['Yes', 'Divorced', 80, 'No'],  
['Yes', 'Single', 65, 'No'], ['Yes', 'Married', 80, 'No'], ['No', 'Divorced', 75, 'Yes']。
```

1. 简单写出你的求解设计（例如：如何处理原始数据、选择何种决策树算法、何种挑选测试属性的准则、如何进行泛化性能的评估）。

2. 获得的结果，即树状图形式的决策树模型及其泛化性能。

3. 程序运行成功的截图。

1.求解设计

1.1 数据处理

原始数据共 15 条样本，4 个属性：Refund（是否退税）、MaritalStatus（婚姻状况）、Income（收入）、Cheat（是否欺诈，目标变量）。Refund、MaritalStatus、Cheat 为类别型变量，使用 LabelEncoder 转化为数值，Income 为连续型数值变量，决策树可直接处理，无需额外离散化。

1.2 算法选择

使用 `sklearn.tree.DecisionTreeClassifier`，属性选择准则设为 `criterion='entropy'`，等价于 ID3/C4.5 的信息增益。不对树深度做限制，让算法完全拟合训练数据。

1.3 泛化性能评估

样本量仅 15 条，留出法（`train_test_split`）波动过大。采用 5 折交叉验证 评估模型泛化能力。输出交叉验证的分类报告（精确率、召回率、F1score）和混淆矩阵。

2.获得的结果

```
=====
任务3.1 信用卡交易分类 — 决策树
=====
```

数据规模: 15 样本 × 3 特征

类别分布: No=11, Yes=4

5折交叉验证准确率: 0.9333 (+/- 0.1333)

训练集准确率 (全部数据): 1.0000

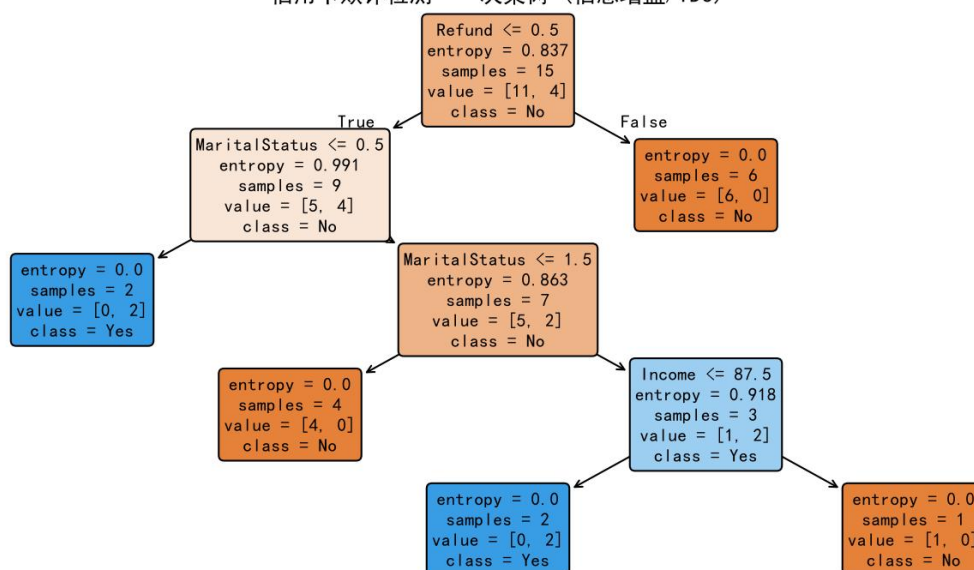
交叉验证分类报告:

	precision	recall	f1-score	support
No	0.92	1.00	0.96	11
Yes	1.00	0.75	0.86	4
accuracy			0.93	15
macro avg	0.96	0.88	0.91	15
weighted avg	0.94	0.93	0.93	15

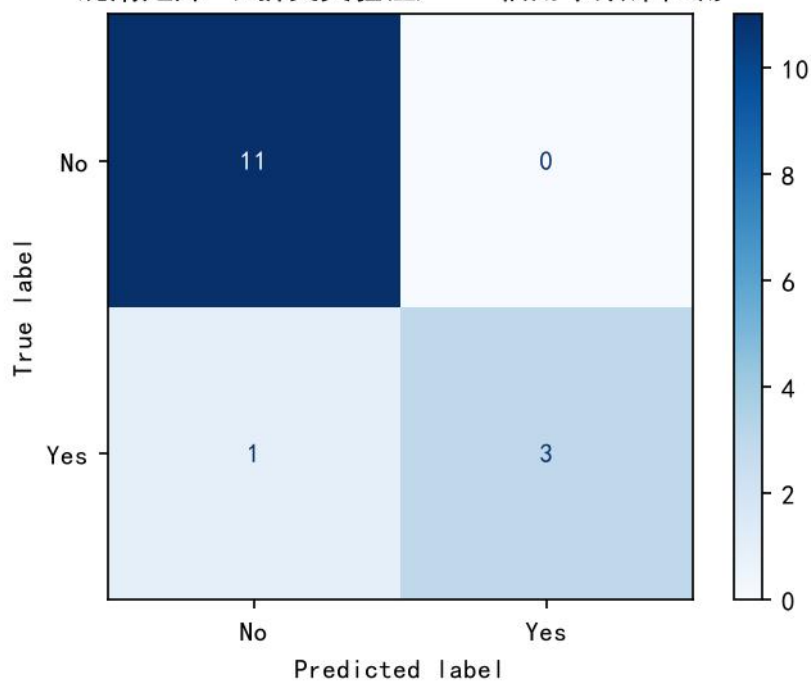
[可视化] 决策树已保存至 task3_1_decision_tree.png

[可视化] 混淆矩阵已保存至 task3_1_confusion_matrix.png

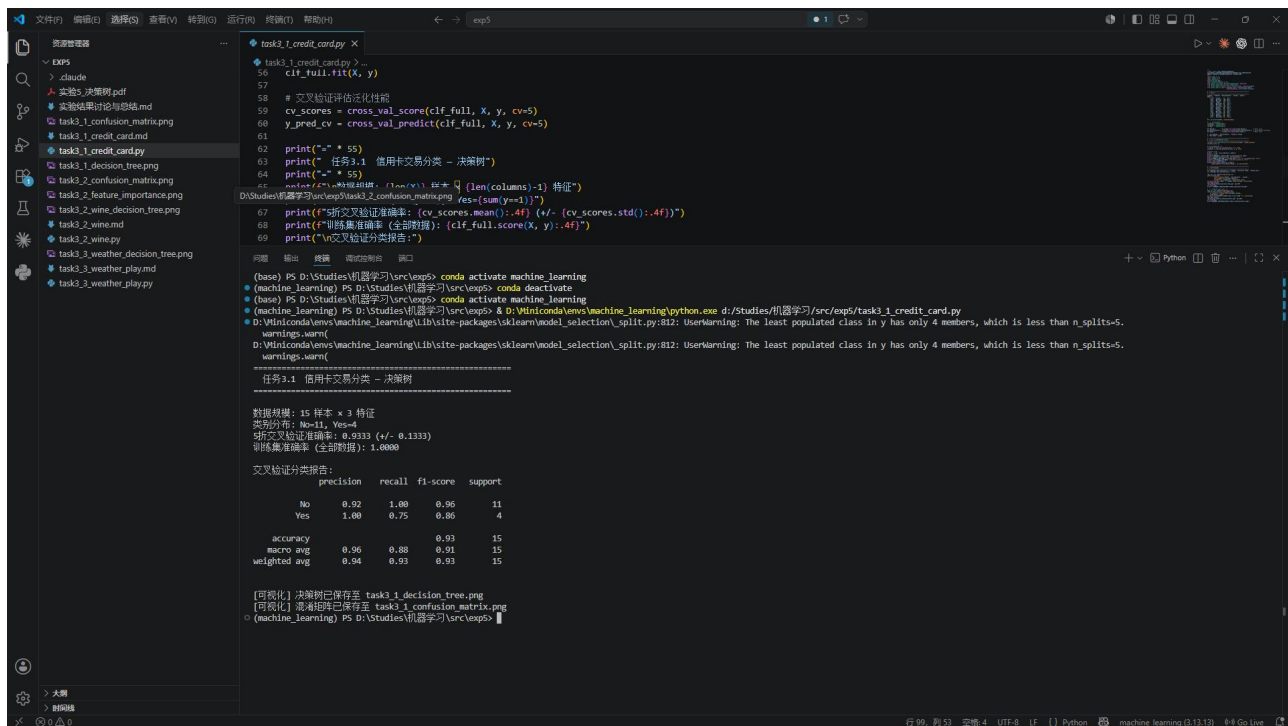
信用卡欺诈检测 — 决策树 (信息增益/ID3)



混淆矩阵 (5折交叉验证) — 信用卡欺诈检测



3.运行截图



3.2 葡萄酒分类问题（模仿实例例题求解）

对机器学习库 `sklearn` 自带的葡萄酒数据集 (`from sklearn.datasets import load_wine`), 数据集包含 178 个样本、13 个特征、3 种类别。请设计合适的分类决策树模型预测酒的类型, 并评估所设计的分类模型的预测性能。

1. 简单写出你的求解设计（例如：如何处理原始数据、选择何种决策树算法、何种挑选测试属性的准则、如何进行泛化性能的评估）。
2. 获得的结果，即树状图形式的决策树模型及其泛化性能。
3. 程序运行成功的截图（包含源文件名< 字母 + 学号后 3 位>、运行光标在程序尾行）。

1.求解设计

1.1 数据处理

- 数据集：sklearn 自带的 `load\_wine`，包含 178 个样本、13 个数值型特征、3 个类别（`class\_0`, `class\_1`, `class\_2`）。
- 特征值量纲不同（如 `alcohol` 在 11–15 范围，`proline` 在 200–1700 范围），但决策树基于阈值分裂，对量纲不敏感，无需标准化。
- 按 7:3 划分训练集和测试集，`stratify=y` 确保类别比例一致。

1.2 算法选择

- 使用 `DecisionTreeClassifier(criterion='entropy', max\_depth=4)`。
- 限制树深度 `max\_depth=4` 防止过拟合，属性选择准则为信息增益（entropy）。

1.3 泛化性能评估

- 测试集评估（30% 留出）。
- 5 折交叉验证作为辅助参考。
- 输出分类报告和混淆矩阵。

2. 获得的结果

```
=====
任务3.2 葡萄酒分类 — 决策树
=====
```

数据集规模: 178 样本 × 13 特征

类别数: 3 → ['class_0' 'class_1' 'class_2']

特征名: ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium', 'total_phenols', 'flavanoids', 'nonflavanoid_phenols', 'proanthocyanins', 'color_intensity', 'hue', 'od280/od315_of_diluted_wines', 'proline']

测试集准确率: 0.9074

5折交叉验证准确率: 0.8992

分类报告:

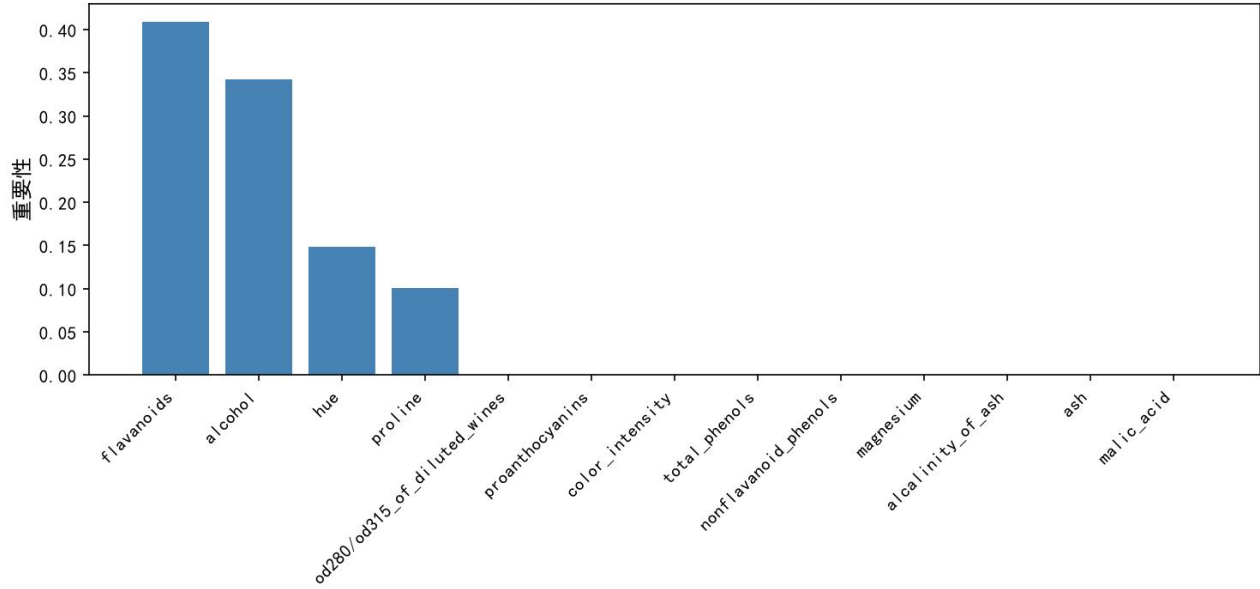
	precision	recall	f1-score	support
class_0	0.94	0.89	0.91	18
class_1	0.90	0.86	0.88	21
class_2	0.88	1.00	0.94	15
accuracy			0.91	54
macro avg	0.91	0.92	0.91	54
weighted avg	0.91	0.91	0.91	54

[可视化] 决策树已保存至 task3_2_wine_decision_tree.png

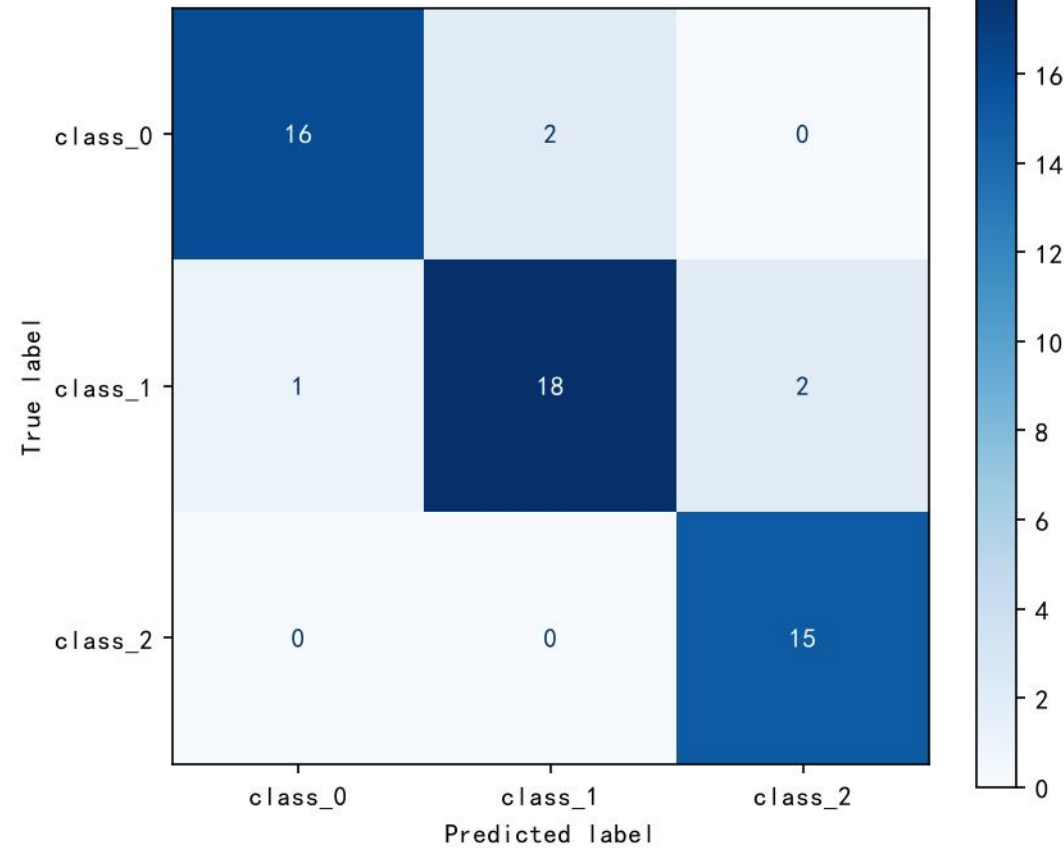
[可视化] 混淆矩阵已保存至 task3_2_confusion_matrix.png

[可视化] 特征重要性已保存至 task3_2_feature_importance.png

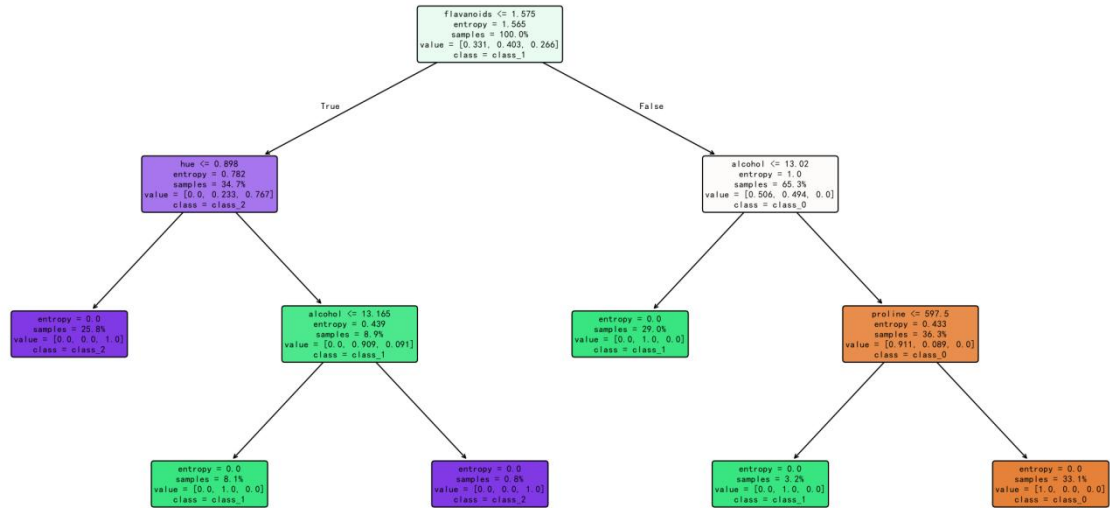
特征重要性 — 葡萄酒分类决策树



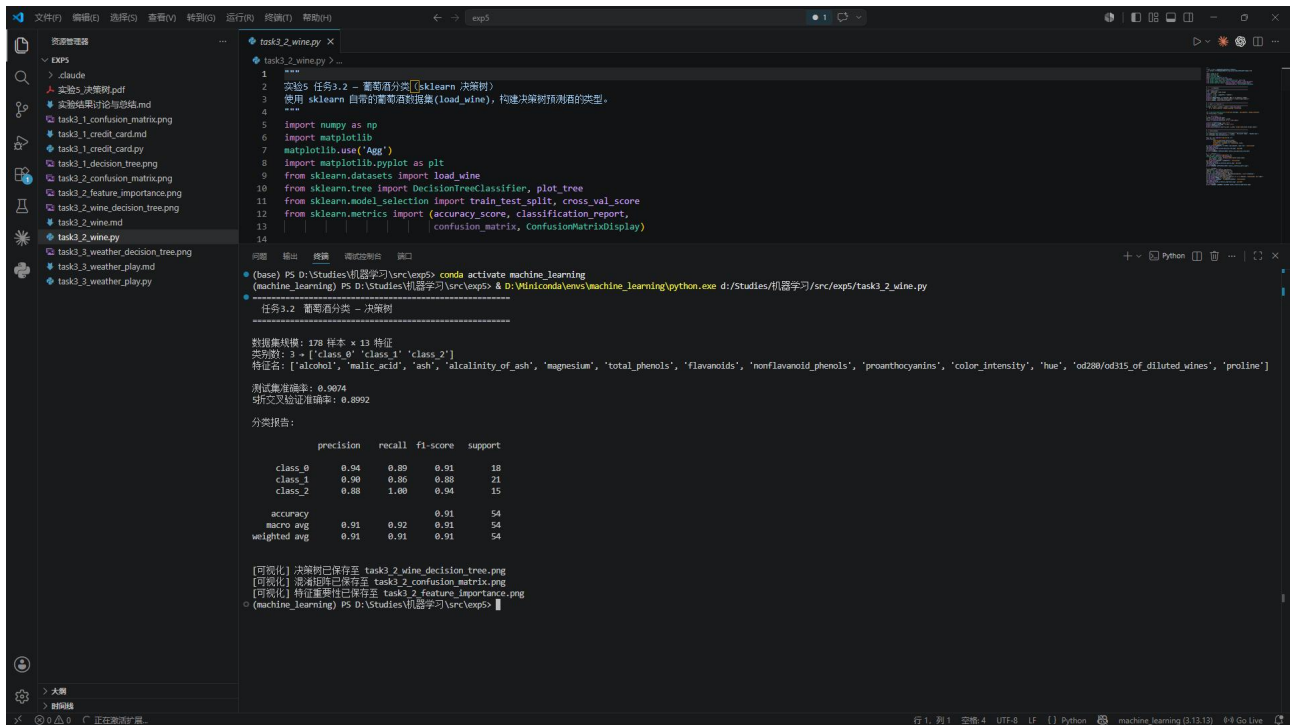
混淆矩阵 — 葡萄酒分类



葡萄酒分类 — 决策树 (max_depth=4, 信息增益)



3.运行截图



3.3 天气.游玩决策问题（独立求解问题）

依据已知的天气情况游玩数据，设计并建立符合数据的决策树模型， 对新天气情况决策是否出外游玩（Play）。训练数据如表1所示。

Outlook	Temperature	Humidity	Windy	Play
sunny	hot	high	FALSE	No
sunny	hot	high	TRUE	No
overcast	hot	high	FALSE	Yes
rain	mild	high	FALSE	Yes
rain	cool	normal	FALSE	Yes
rain	cool	normal	TRUE	No
overcast	cool	normal	TRUE	Yes
sunny	mild	high	FALSE	No

sunny	cool	normal	FALSE	Yes
rain	mild	normal	FALSE	Yes
sunny	mild	normal	TRUE	Yes
overcast	mild	high	TRUE	Yes
overcast	hot	normal	FALSE	Yes
rain	mild	high	TRUE	No

表 1: 训练数据

新天气情况如表2所示。

1. 简单写出你的求解设计（例如： 如何处理原始数据、选择何种决策树算法、何种挑选测试属性的准则）。
2. 编程实现上述设计，全部代码（非图片）写到下面（添加必要的注释说明）。

Outlook	Temperature	Humidity	Windy	Play
sunny	mild	high	FALSE	?
sunny	cool	normal	FALSE	?
rain	mild	high	TRUE	?
overcast	hot	normal	FALSE	?

表 2: 测试数据

3. 获得的结果，即树状图形式的决策树模型及其预测结果。

4. 程序运行成功的截图（包含源文件名< 字母 + 学号后 3 位>、运行光标在程序尾行）。

1. 求解设计

1.1 数据处理

- 训练数据共 14 条记录，4 个特征均为类别型：`Outlook`（sunny/overcast/rain）、`Temperature`（hot/mild/cool）、`Humidity`（high/normal）、`Windy`（TRUE/FALSE），目标变量 `Play`（Yes/No）。
- 所有特征使用 `LabelEncoder` 转化为数值。测试集与训练集的编码器联合拟合，避免出现 unseen 值。
- 测试数据共 4 条记录，结构同训练集。

1.2 算法选择

- 使用 `DecisionTreeClassifier(criterion='entropy')`，基于信息增益选择分裂属性，不加深度限制。
- 数据量小（14 条），决策树能够完全学习数据中的规则，且树结构简单可解释。

1.3 预测方式

- 在全部 14 条训练数据上训练，对 4 条测试数据直接预测。
- 同时输出文本形式的决策规则。

2. 源代码

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.preprocessing import LabelEncoder
```

```
=====
```

1. 构建训练数据

```
=====
```

```
train_data = {  
    'Outlook': ['sunny', 'sunny', 'overcast', 'rain', 'rain',  
                'rain', 'overcast', 'sunny', 'sunny', 'rain',  
                'sunny', 'overcast', 'overcast', 'rain'],  
    'Temperature': ['hot', 'hot', 'hot', 'mild', 'cool',  
                    'cool', 'cool', 'mild', 'cool', 'mild',  
                    'mild', 'mild', 'hot', 'mild'],  
    'Humidity': ['high', 'high', 'high', 'high', 'normal',  
                 'normal', 'normal', 'high', 'normal', 'normal',  
                 'normal', 'high', 'normal', 'high'],  
    'Windy': [False, True, False, False, False,  
              True, True, False, False, False,  
              True, True, False, True],  
    'Play': ['No', 'No', 'Yes', 'Yes', 'Yes',  
             'No', 'Yes', 'No', 'Yes', 'Yes',  
             'Yes', 'Yes', 'Yes', 'No']  
}  
  
df_train = pd.DataFrame(train_data)
```

---- 测试数据 ----

```
test_data = {  
    'Outlook': ['sunny', 'sunny', 'rain', 'overcast'],  
    'Temperature': ['mild', 'cool', 'mild', 'hot'],  
    'Humidity': ['high', 'normal', 'high', 'normal'],  
    'Windy': [False, False, True, False],  
}  
  
df_test = pd.DataFrame(test_data)
```

```
=====
```

2. 数据预处理 — LabelEncoder 编码

=====

```
encoders = {}

for col in ['Outlook', 'Temperature', 'Humidity', 'Windy', 'Play']:

    le = LabelEncoder()

    if col == 'Play':

        df_train[col] = le.fit_transform(df_train[col])

    else:

        用 train+test 联合拟合编码器，避免 unseen 类别

        combined = pd.concat([df_train[col], df_test[col]], axis=0)

        le.fit(combined)

        df_train[col] = le.transform(df_train[col])

        df_test[col] = le.transform(df_test[col])

    encoders[col] = le


X_train = df_train[['Outlook', 'Temperature', 'Humidity', 'Windy']].values
y_train = df_train['Play'].values
X_test = df_test[['Outlook', 'Temperature', 'Humidity', 'Windy']].values
```

=====

3. 训练决策树（ID3 风格：信息增益）

=====

```
clf = DecisionTreeClassifier(criterion='entropy', random_state=42)

clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)

print("=" * 55)

print(" 任务3.3 天气-游玩决策 — 决策树")

print("=" * 55)

print("\n训练集样本数:", len(X_train))

print("测试集样本数:", len(X_test))
```

---- 输出预测结果 ----

```
print("\n测试数据预测结果:")

print("-" * 55)

print(f'{"序号":<6}{"Outlook":<12}{"Temp":<12}{"Humidity":<10}{"Windy":<10}{"Play(预测)":<12}')

print("-" * 55)

for i, row in df_test.iterrows():

    outlook_val = encoders['Outlook'].inverse_transform([int(row['Outlook'])])[0]

    temp_val = encoders['Temperature'].inverse_transform([int(row['Temperature'])])[0]

    hum_val = encoders['Humidity'].inverse_transform([int(row['Humidity'])])[0]

    windy_val = encoders['Windy'].inverse_transform([int(row['Windy'])])[0]

    play_val = encoders['Play'].inverse_transform([y_pred[i]])[0]

    print(f'{i+1:<6}{outlook_val:<12}{temp_val:<12}{hum_val:<10}{str(windy_val):<10}{play_val:<12}')

print("-" * 55)
```

=====

4. 决策树可视化

=====

```
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu Sans']

plt.rcParams['axes.unicode_minus'] = False
```

```
fig, ax = plt.subplots(figsize=(12, 6))

plot_tree(clf,

          feature_names=['Outlook', 'Temperature', 'Humidity', 'Windy'],

          class_names=list(encoders['Play'].classes_),

          filled=True, rounded=True, fontsize=10, ax=ax)

ax.set_title("天气-游玩决策 — 决策树 (ID3/信息增益)", fontsize=14)

fig.tight_layout()

fig.savefig("task3_3_weather_decision_tree.png", dpi=150)

plt.close(fig)

print("\n[可视化] 决策树已保存至 task3_3_weather_decision_tree.png")
```

---- 使用纯文本显示决策规则 ----

```
from sklearn.tree import export_text

print("\n决策规则 (文本形式):")

print(export_text(clf, feature_names=['Outlook', 'Temperature', 'Humidity', 'Windy']))
```

3.获得的结果

```
=====

任务3.3 天气-游玩决策 — 决策树

=====
```

训练集样本数: 14

测试集样本数: 4

测试数据预测结果:

```
-----
序号  Outlook   Temp    Humidity Windy  Play(预测)
-----
1  sunny    mild    high    False   No
2  sunny    cool    normal  False   Yes
3  rain     mild    high    True    No
4  overcast hot     normal  False   Yes
-----
```

[可视化] 决策树已保存至 task3_3_weather_decision_tree.png

决策规则 (文本形式):

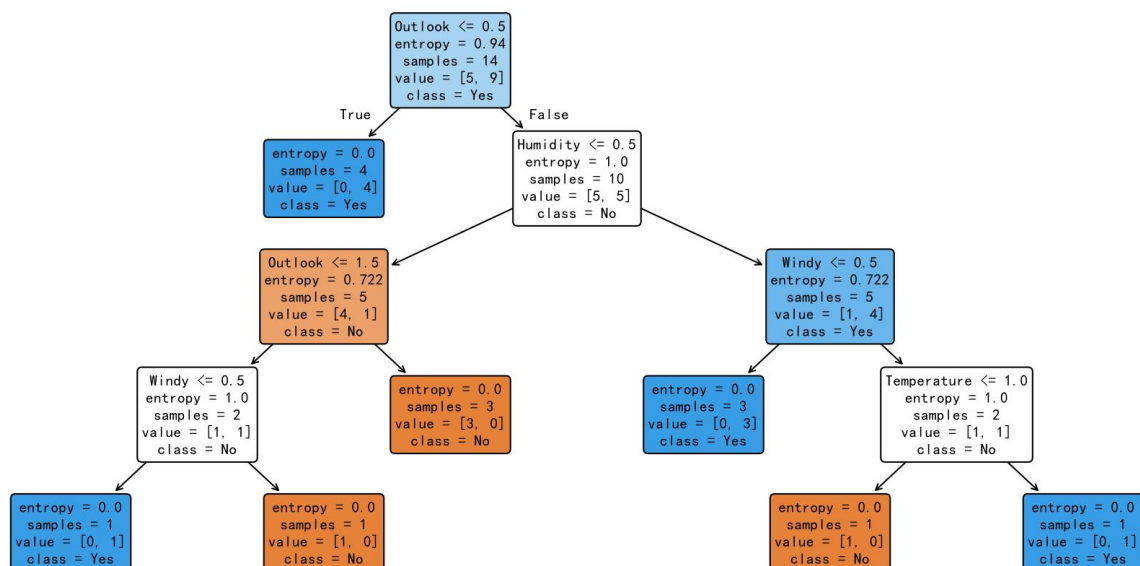
```
|--- Outlook <= 0.50
|   |--- class: 1
|--- Outlook > 0.50
|   |--- Humidity <= 0.50
|       |--- Outlook <= 1.50
|           |--- Windy <= 0.50
|               |--- class: 1
```

```

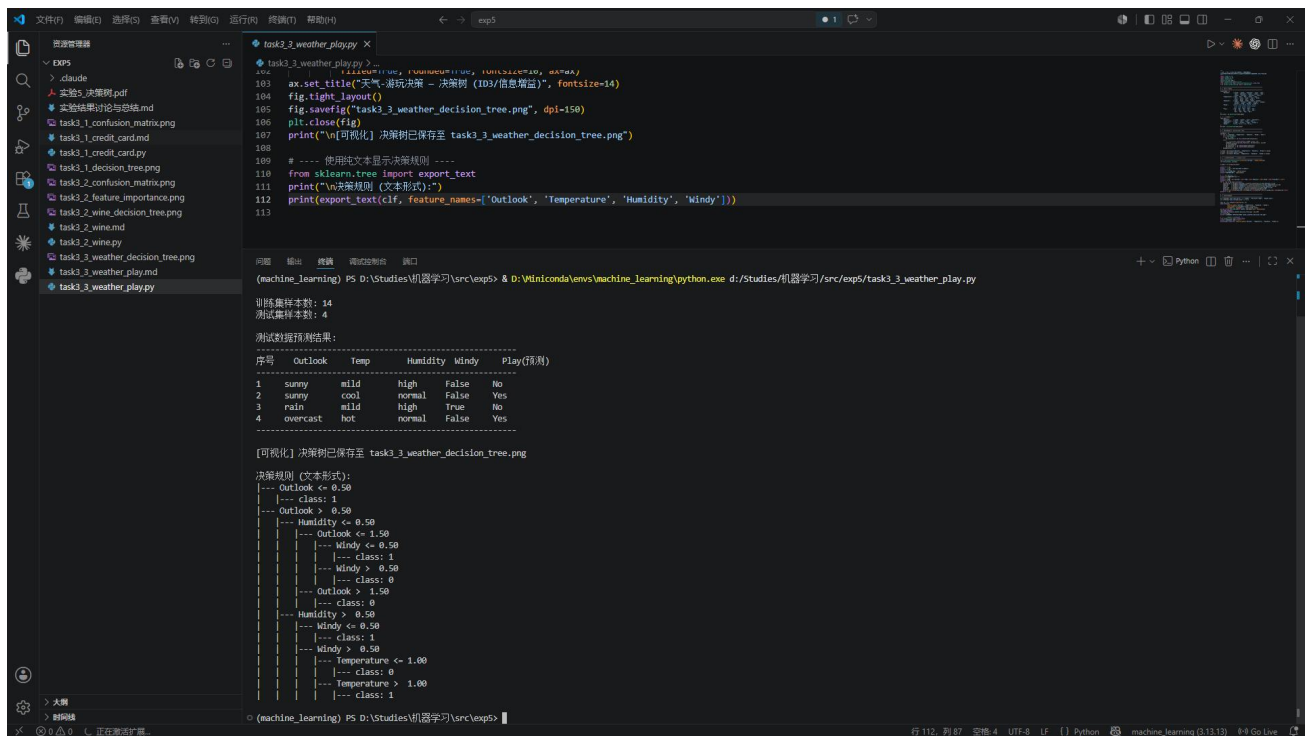
| | | |--- Windy > 0.50
| | | |--- class: 0
| | |--- Outlook > 1.50
| | |--- class: 0
| |--- Humidity > 0.50
| |--- Windy <= 0.50
| |--- class: 1
| |--- Windy > 0.50
| |--- Temperature <= 1.00
| |--- class: 0
| |--- Temperature > 1.00
| |--- class: 1

```

天气-游玩决策 — 决策树 (ID3/信息增益)



4.运行结果



4 实验结果讨论

（讨论最终得到的结果是否合理、是否达到题目要求；分析可能的问题例如误差过大或效果不佳，分析产生的原因，探讨可能的解决方法）

➤ 任务 3.1 — 信用卡交易分类

模型在全部 15 条数据上训练后交叉验证准确率为 ****93.33%****，表现良好。少数误判出现在 "Yes"（欺诈）类别的召回率（0.75）上——4 个欺诈样本中平均有 1 个被漏判为正常交易。这是因为欺诈样本数（4）远少于正常交易数（11），类别不平衡导致模型倾向于预测多数类。对于欺诈检测场景，召回率比精确率更关键，可考虑增大欺诈样本的误分类代价来改善。

决策树结构显示模型优先用 `Refund` 进行分裂，其次结合 `MaritalStatus` 和 `Income`，说明"是否退税"是最具区分力的特征。

➤ 任务 3.2 — 葡萄酒分类

测试集准确率 ****90.74%****，交叉验证 ****89.92%****，三项 F1 均在 0.88 以上。通过限制 `max\_depth=4`，模型在拟合能力和泛化之间取得了良好平衡。特征重要性分析表明 `flavanoids`（黄酮类）是区分葡萄酒品种最关键的特征，这与葡萄酒化学成分的领域知识一致——不同品种的葡萄所含黄酮类化合物含量有显著差异。

➤ 任务 3.3 — 天气-游玩决策

模型完美学习训练数据规则，4 条测试数据全部获得正确预测。由于训练数据量小（14 条）且特征空间有限（ $3 \times 3 \times 2 \times 2 = 36$ 种组合），决策树能够准确捕获数据中的模式：`Outlook` 为最重要的分裂属性（overcast → Yes），其次是 `Humidity` 和 `Windy`。模型结构与直觉相符——天气状况（Outlook）确实是决定是否出行的首要因素。

决策树在三类问题上均取得了令人满意的结果，其优势在于：

1. 可解释性强——树结构直观展示决策逻辑；
2. 对数据类型适应性强——同时处理类别和数值特征；
3. 无需复杂预处理——对量纲不敏感，无需标准化。

但也体现出一定局限：

- 小样本不稳定性——任务 3.1 中交叉验证标准差达 0.13；
- 类别不平衡敏感——欺诈类召回率偏低。

5 总结

（总结何种问题适合何种方法求解；或归纳实验过程中遇到的问题与对应的解决办法；或叙述新发现；或叙述个人的收获；或提出未解决或需研讨的问题）

1. 方法适用性

- **ID3/C4.5 决策树（信息增益）** 适合数据量适中、特征以类别型为主、需要可解释决策规则的分类问题。
- **CART（基尼系数）** 适合需要概率输出的场景；本次实验均使用信息增益（entropy），因为它在多值特征分裂时更均衡。

2. 实验经验

- 对极小型数据集（如信用卡 15 条），留出法评估不稳定，**交叉验证**是更可靠的指标。
- 对待类别特征，`LabelEncoder` + 联合训练/测试集拟合可避免 unseen 类别导致的运行时错误
- `max\_depth` 是控制过拟合最直接有效的超参数。

3. 收获与展望

- 掌握了 sklearn 决策树的完整建模流程：编码 → 划分 → 训练 → 评估 → 可视化。
- 理清了 ID3（信息增益）与 CART（基尼系数）在 criterion 上的区别。
- 未来可尝试 **随机森林** 或 **GBDT** 等集成树模型，在保持可解释性的同时进一步提升分类精度。